



SQL

数据的域类型

- char(n): 定长字符串
- varchar(n): 可变长字符串 (最大n)
- int
- smallint
- numeric(p,d): 定点数, p为总位数, d为小数位数
- real: 浮点数double
- float(n): 至少n位精度的浮点数
- null: 是否可为空值
- date: yyyy-mm-dd
- time: hh:mm:ss
- timestamp: date+time
- interval: 时间段
- lob: 大对象 (照片、视频、大段文字等), 查询返回的指针

建表语句

```
create table  $r$ 
( $A_i$   $D_i$  (integrity_constraint),
...,
(integrity_constraintk),
...)
```

- r : 表名
- A_i : 属性名
- D_i : 数据类型
- *integrity_constraint*: 完整性约束
 - 域约束: 可以从已有数据类型创建新的域
 - create domain hourly_wage numeric(5, 2)
 - not null: 非空
 - unique: 不允许重复 (允许null)
 - primary key: 主键 (不允许null)

- 跟在一个属性后面表示单主键
- 单列一行且primary key ($A_1, \dots, A_n$)表示组合主键
- default: 默认值
- foreign key($A_{k1}, \dots, A_{kn}$) references s : 外键 (若无指定, 默认引用主键)
- check (P): 检查是否满足约束P
- 以上约束均可对创建的域进行限制
 - create domain hourly_wage numeric(5, 2)
 - constraint value_test check(value >= 6.00) not null

参照完整性约束

子集依赖

- 设关系 $r_1(R_1), r_2(R_2)$, 主键分别为 K_1, K_2
- 若 $\alpha \in R_2$ 是引用 r_1 的外键
- 则对于 r_2 中的每个元组 t_2 , 必须存在 $t_1 \in r_1$, 使得 $t_1[K_1] = t_2[\alpha]$
- 即 $\Pi_{\alpha}(r_2) \subseteq \Pi_{K_1}(r_1)$

插入、删除、更新规则

- 插入: 需满足上述子集依赖
- 删除: 从 r_1 删除 t_1 , 系统查找引用它的元组: $\sigma_{\alpha=t_1[K]}(r_2)$, 若该集合非空:
 - 拒绝删除
 - 删除该集合所有的元组 (级联删除)
- 更新:
 - 若更新了 r_2 中的外键 α , 则需验证对应被引用的键是否存在
 - 若更新了 r_1 中被引用的键, 则需查找 r_1 中引用旧值的元组进行处理
 - 拒绝更新 r_1
 - 级联更新 r_2
 - 删除这些元组
- 通过cascade语句来设置级联更新/删除
 - on delete cascade
 - on update cascade
 - 若多关系之间存在外键链条且指定了on delete cascade, 则一个端点的删除会在整条链上传播
- 参照完整性只在事务结束时进行, 事务中间允许暂时违反

断言

create assertion <assertion-name> check <predicate>

- 创建断言时，系统会测试其有效性
- 在每次可能违反断言的更新后再次测试
- 违反断言的更新操作会被拒绝

插入语句

insert into r values (...)

- 违反完整性约束会被拒绝
- 还可以通过查询构造新元组

insert into r select A_1, A_2 from s (where ...)

删除语句

delete from r (where ...)

- 从 r 中删除所有/满足...条件的元组

drop table r

- 删除表 r

更新语句

update r set ... (where ...)

对于多种情况的更新，用case语句

```
update account
  set balance =
  case
    when balance <= 10000 then balance * 1.05
    else balance * 1.06
  end
```

```
case
  when  $pred_1$  then  $result_1$ 
  when  $pred_2$  then  $result_2$ 
  ...
  when  $pred_n$  then  $result_n$ 
  else  $result_0$ 
end
```

调整语句

alter table r add/drop A

- 在 r 中加入/删除属性 A

select语句

对应投影操作

select $A_1, \dots, A_n$ from r 等价于 $\Pi_{A_1, \dots, A_n}(r)$

- distinct: 去重
- all: 不去重 (默认)
- 用 $*$ 代替 A_i : 查询所有属性
- 支持算术运算: 对应广义投影

from语句

from r_1, r_2

对应笛卡尔积 $r_1 \times r_2$

from后的表可以是各种形式的

where语句

where P

对应 $\sigma_P(r)$

- 比较条件可以用and、or、not组合
- 支持between

natural join语句

r_1 natural join r_2 natural join ... natural join r_n

对应 $r_1 \bowtie r_2 \bowtie \dots \bowtie r_n$

join using/on语句

r_1 join r_2 using $(A_1, \dots, A_n)$

- 在 $A_1, \dots, A_n$ 上做连接

$r_1 \text{ join } r_2 \text{ on } (r_1.A_1 = r_2.B_2, \dots, r_1.A_n = r_2.B_n)$

- 连接 r_1, r_2 , 条件是 $r_1.A_1 = r_2.B_2, \dots, r_1.A_n = r_2.B_n$

inner/(left/right/full) outer join与关系代数中的相同

更名操作

对属性/表均可更名

`select  $rs.A$  as  $B$  from  $r$  natural join  $s$  as  $rs$`

字符串操作

- %: 替代任意长的字符串
- _: 替代一个字符
- (not) like: 模式匹配
- ||: 字符串连接
- 大小写转换:
 - `upper('text') → 'TEXT'`
 - `lower('TEXT') → 'text'`
- 字符串函数:
 - `length('abc') → 3`
 - `substring('Hello', 2, 3) → 'ell'`

排序操作

`order by  $A$  desc/asc`

按属性 A 排序

- desc: 降序
- asc: 升序 (默认)

还可以先按 A 排再按 B 排 (A 相等时)

集合操作

union、intersect和 except, 对应 $\cup$ 、 $\cap$ 和 $-$

默认去除重复元组 (使用union/intersect/except all来避免去重)

若元组在关系 r 中出现 m 次, 在 s 中出现 n 次, 则:

- union all: 出现 $m + n$ 次
- intersect all: 出现 $\min(m, n)$ 次
- except all: 出现 $\max(0, m - n)$ 次

null

与关系代数中对null的处理相同

- is null返回true
- 聚合函数中count(*)会统计null

聚合函数

- avg: 平均值
- min: 最小值
- max: 最大值
- sum: 求和
- count: 计数
 - count(A): 统计 A 有值的元组数 (排除null)
 - count(*): 统计所有元组数 (包括null)
 - count(distinct A): 统计 A 不同取值的个数
- group by A
 - 按 A 分组后再计算/统计
 - 和聚合函数一起出现在select语句中的属性必须出现在group by中
- having (P)
 - 统计满足条件 P 的元组
 - 聚合函数中的where
 - where是分组前筛选
 - having是分组后筛选

嵌套子查询

嵌套在另一个查询中的 select-from-where 表达式

- 集合成员资格测试
 - in/not in

```
-- 查询既有账户又有贷款的客户
select distinct customer_name
from borrower
where customer_name in (select customer_name
                        from depositor)

-- 查询有贷款但没有账户的客户
select distinct customer_name
from borrower
where customer_name not in (select customer_name
                           from depositor)
```

- 集合比较（数学比较）<, <=, >=, =, <>
 - > some/any: 大于子查询结果中的至少一个值
 - > all: 大于子查询结果中的所有值
 - (= some) == in
 - (!= all) == not in

```
-- 查询资产大于布鲁克林某些分行的所有分行
select distinct T.branch_name
from branch as T, branch as S
where T.assets > S.assets and
      S.branch_city = 'Brooklyn'

-- 使用>some子句的相同查询
select branch_name
from branch
where assets > some
      (select assets
       from branch
       where branch_city = 'Brooklyn')
```

- 存在性测试
 - exists: 子查询返回至少一行时结果为真
 - not exists: 子查询不返回任何行时结果为真
 - 常用于关系包含性测试(如"not exists (B except A)"表示A包含B)
- 唯一性测试
 - unique: 测试子查询结果是否没有重复行
 - not unique: 测试子查询结果是否有重复行
 - 用于检查基数约束(如"最多一个"、"至少两个")

```
-- 查询在嘉定分行至少有两个账户的客户
select distinct T.customer_name
from depositor T
where not unique(
    select R.customer_name
    from account, depositor as R
    where T.customer_name = R.customer_name and
           R.account_number = account.account_number and
           account.branch_name = 'Jiading')
```

视图

create view *v* as <query expression>

- <query expression>: 合法的SQL查询表达式
- 存储方式:
 - 视图定义(查询表达式)存储在数据库中
 - 实际数据不单独存储, 使用时动态计算
- 使用方式:
 - 视图名称可以像普通表一样在查询中使用
 - 查询视图时, 数据库系统会将视图定义替换为实际的查询表达式
- 内联视图: 一次性的, 直接把<query expression>写在需要用到的地方

with语句

with cte_name (column1, column2,...) as (select ...)

select ... from cte_name ...

- 仅在当前查询生效（相当于临时视图）
- 同一个with子句中定义的cte可以在后续cte中引用
- 支持递归cte

通过视图的更新

限制条件

- select子句中
 - 只能包含基表的直接属性名
 - 不能包含
 - 表达式或计算
 - 聚合函数
 - distinct
- from子句中
 - 只能包含单个基表
 - 不能包含join或子查询
- 不能包含group by或having
- 隐藏属性必须可以为null或有默认值

通过insert语句，将新的元组加入视图的基表中，隐藏属性取null/默认值

物化视图

存储特性：

- 物理存储数据，像普通表一样占用存储空间
- 预先计算：创建时执行查询并存储结果

更新机制：指的是基表变化时更新视图

- 实时更新：基表事务提交时
- 定时更新：固定时间间隔或手动触发

事务

作为单个单元执行的一系列查询和更新语句序列

隐式开始，显式终止：

- commit [work]: 使事务的所有更新永久生效到数据库
- rollback [work]: 撤销事务执行的所有更新
 - 未完成/在中间某步中断的事务会自动回滚

索引

create index <name> on <relation-name> (attribute)

- 在查询时节省空间/时间

权限

数据操作权限

- select: 允许读取
- insert: 允许插入
- update: 允许修改
- delete: 允许删除

模式修改权限

- index: 允许创建/删除索引
- resource: 允许创建表
- alteration: 允许增加/删除属性
- drop: 允许删除表

对于视图

- 创建视图无需授权
- 对视图的权限与对基表权限相同

授权图

授权从一个用户传递到另一个用户的过程可以用授权图表示:

- 节点代表用户
- 根节点是数据库管理员(DBA)
- 授权收回会沿着边的方向传递

角色

- 允许我们为一类用户统一指定常见的权限
- 权限可以被授予 (grant) 或撤销 (revoke) 角色
- 角色可以被赋予给用户，甚至可以赋予给其他角色

create role <role name>

授权/撤销语句

grant/revoke <privilege list>

on <relation name or view name>

to/from <user list> (with grant option/restrict)

user list可以是：

- 用户ID
- public
- 角色

with grant option：允许被授权用户将权限转授给其他用户

restrict：防止级联撤销

审计日志

对数据库所有更改（插入 / 删除 / 更新）的记录

- 哪个用户执行了更改
- 更改是在什么时候执行的

可通过触发器实现

数据加密

数据加密标准 (DES)

通过加密密钥对字符进行替换并重新排列其顺序

密钥通过安全机制提供给授权用户

该方案的安全性不高于密钥传输机制的安全性，因为密钥需要共享

高级加密标准 (AES)

是替代 DES 的新标准，基于 Rijndael 算法，也依赖共享的秘密密钥

公钥加密 (Public-key encryption)

每个用户拥有两个密钥：

- 公钥 (public key)：用于加密数据，但不能用于解密数据
- 私钥 (private key)：用于解密数据